

UD4.OPTIMIZACIÓN Y DOCUMENTACIÓN

4.2. REFACTORIZACIÓN

INTRODUCCIÓN

La **refactorización** es una técnica de Ingeniería del software que permite la optimización de un código previamente escrito, por medio de cambios en su estructura interna sin que esto suponga alteraciones en su comportamiento externo. Al refactorizar, los desarrolladores pueden eliminar el código redundante, mejorar las convenciones de nomenclatura y mejorar la calidad general del código.

La **refactorización** busca mejorar la comprensión del código para facilitar nuevos desarrollos, resolución de errores o adición de nuevas funcionalidades.

- ▶ Limpia el código, mejorando consistencia y claridad.
- ▶ Mantiene el código, no corrige errores ni añade funciones nuevas.
- ▶ Permite la realización de cambios en el código.
- ▶ Se obtiene un código limpio y altamente modularizado.

INTRODUCCIÓN

```
public abstract class AbstractCollection implements Collection {  
    public void addAll(AbstractCollection c) {  
        if (c instanceof Set) {  
            Set s = (Set)c;  
            for (int i=0; i < s.size(); i++) {  
                if (!contains(s.getElementAt(i))) {  
                    add(s.getElementAt(i));  
                }  
            }  
        }  
        else if (c instanceof List) {  
            List l = (List)c;  
            for (int i=0; i < l.size(); i++) {  
                if (!contains(l.get(i))) {  
                    add(l.get(i));  
                }  
            }  
        }  
        else if (c instanceof Map) {  
            Map m = (Map)c;  
            for (int i=0; i<m.size(); i++)  
                add(m.keys[i], m.values[i]);  
        }  
    }  
}
```

Duplicated Code

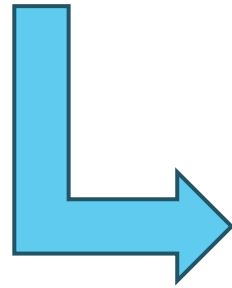
Duplicated Code

Alternative Classes with Different Interfaces

Switch Statement

Inappropriate Intimacy

Long Method



```
import java.util.*;  
  
public abstract class AbstractCollection<T> implements Collection<T> {  
  
    public void addAll(AbstractCollection<T> c) {  
        for (T element : c) {  
            if (!contains(element)) {  
                add(element);  
            }  
        }  
    }  
  
    // Métodos abstractos que deben ser implementados por subclases  
    protected abstract boolean contains(T element);  
    protected abstract void add(T element);  
}
```

¿CUÁNDO REFACTORIZAR?

La **refactorización** se debe ir haciendo mientras se va realizando el desarrollo de la aplicación. Síntomas que indican que es necesaria (bad smells):

- ▶ Código duplicado: Detección del mismo código en distintos lugares, hay que extraerlo y unificarlo.
- ▶ Métodos muy largos: Descomponer en otros más pequeños. En POO cuanto más corto es un método más fácil es reutilizarlo.
- ▶ Clases muy grandes: Si una clase intenta resolver muchos problemas tiene demasiados métodos y atributos. Hay que intentar crear clases más pequeñas, con un conjunto de responsabilidades limitadas.
- ▶ Lista de parámetros extensa: En la POO no se suele pasar muchos parámetros a los métodos, sólo los mínimamente necesarios. Demasiados parámetros generan un problema de encapsulación de datos o la necesidad de crear una clase de objetos a partir de esos parámetros y pasar el objeto como argumento.

¿CUÁNDO REFACTORIZAR?

- ▶ Cambio divergente: Si una clase es modificada por diversos motivos, no relacionados entre ellos, lo mejor es eliminar la clase.
- ▶ Cirugía a tiro pistola: Síntoma que se presenta cuando después de un cambio a una determinada clase, hay que realizar modificaciones en diversos lugares del código.
- ▶ Envidia de funcionalidad: Síntoma que se presenta cuando un método utiliza más cantidad de elementos de otra clase que de la suya propia. Se soluciona cambiando el método de clase.
- ▶ Clase de solo datos: Clase que sólo tiene atributos y métodos de acceso a ellos. ¿Son necesarias?
- ▶ Legado rechazado: Subclases que utilizan sólo unas pocas características de sus superclases. Jerarquía de clases incorrecta.

REFACTORIZACIÓN EN ECLIPSE

ECLIPSE(*) tiene distintos métodos (patrones) de refactorización:

- ▶ Rename: Cambia el nombre de variables, clases, métodos, paquetes, directorios y casi cualquier identificador Java.
- ▶ Move: Mueve una clase de un paquete a otro, se mueve el archivo .java a la carpeta y se cambian todas las referencias.
- ▶ Extract Constant: Convierte un número o cadena literal en una constante. Se muestran los cambios y el antes y el después.
- ▶ Extract Local Variable: Asignar una expresión a una variable local. Cualquier referencia a la expresión en el ámbito local se sustituye por la variable. Se muestran los cambios y el antes y el después.
- ▶ Convert Local Variable to Field: Convierte una variable local en un atributo privado de la clase.
- ▶ Inline: consiste en eliminar variables, métodos o clases innecesarios, reemplazándolos directamente por su valor o expresión. Se usa cuando una variable o método intermedio no aporta claridad y solo añade complejidad al código.

REFACTORIZACIÓN EN ECLIPSE

- ▶ Extract Method: Permite seleccionar un bloque de código(con llaves cerradas) y convertirlo en un método. Muy útil para fragmentar métodos largos y para crear un método de código repetido frecuentemente.
- ▶ Change Method Signature: Cambia la firma (el nombre y los parámetros) de un método. Si se cambia el tipo de dato, aparecerán errores de compilación.
- ▶ Member Type to Top Level: Convierte una clase anidada en una clase de nivel superior con su propio archivo de java. Si la clase no es estática, solicita un nombre.
- ▶ Extract Interface: Crea una Interface (plantilla que define qué puede y qué no puede hacer una clase) a partir de métodos de una clase.
- ▶ Extract Superclass: Permite extraer una superclase, seleccionando los métodos y atributos. Puede haber fallos de compilación.
- ▶ Convert Anonymous Class to Nested: Convierte una clase anónima(sin nombre de la que sólo se crea un objeto) en una clase anidada de la clase que lo contiene.

Ejemplo: eliminar código duplicado

```
public class CalculadoraAreas {  
    public double calcularAreaRectangulo(double base, double altura) {  
        return base * altura;  
    }  
  
    public double calcularAreaTriangulo(double base, double altura) {  
        return (base * altura) / 2;  
    }  
  
    public double calcularAreaParalelogramo(double base, double altura) {  
        return base * altura;  
    }  
}
```

- Eliminamos código duplicado ya que calcularBasePorAltura encapsula la operación común.
- Código más limpio y mantenible: Si cambia la forma de calcular $\text{base} * \text{altura}$, solo lo hacemos en un lugar.
- Mejor reutilización: Se evita repetir la lógica en cada método.

Observamos que la lógica de multiplicación ($\text{base} * \text{altura}$) se repite.



```
public class CalculadoraAreas {  
    private double calcularBasePorAltura(double base, double altura) {  
        return base * altura;  
    }  
  
    public double calcularAreaRectangulo(double base, double altura) {  
        return calcularBasePorAltura(base, altura);  
    }  
  
    public double calcularAreaTriangulo(double base, double altura) {  
        return calcularBasePorAltura(base, altura) / 2;  
    }  
  
    public double calcularAreaParalelogramo(double base, double altura) {  
        return calcularBasePorAltura(base, altura);  
    }  
}
```


Ejemplo: extraer constante

```
public class Calculadora {  
    public double calcularAreaCirculo(double radio) {  
        return 3.1416 * radio * radio;  
    }  
  
    public double calcularCircunferencia(double radio) {  
        return 2 * 3.1416 * radio;  
    }  
}
```

Si deseamos alterar este valor que se repite en varios lugares, hay que ir uno por uno.



- ▶ Código más legible: PI es más claro que 3.1416
- ▶ Fácil mantenimiento: Si cambia el valor de PI, solo se modifica en un lugar.
- ▶ Evita errores: Se reduce el riesgo de escribir valores incorrectos en diferentes partes del código.

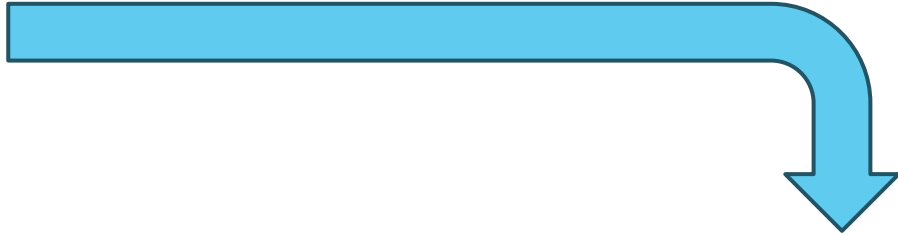
```
public class Calculadora {  
    private static final double PI = 3.1416;  
    // Se extrae la constante  
  
    public double calcularAreaCirculo(double radio) {  
        return PI * radio * radio;  
    }  
  
    public double calcularCircunferencia(double radio) {  
        return 2 * PI * radio;  
    }  
}
```

Ejemplo: inline

```
public class CalculadoraInline {  
  
    public double calcularIVA(double precio) {  
        return precio * 0.16;  
    }  
  
    public double calcularPrecioFinal(double precio) {  
        return precio + calcularIVA(precio);  
    }  
  
}
```

Si un método solo delega una operación simple, podemos reemplazarlo directamente en su uso.

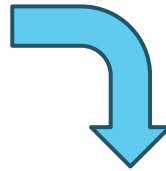
- ▶ El código ahora es más simple y fácil de leer.
- ▶ Hay menos sobrecarga de llamadas innecesarias.
- ▶ Hay mayor eficiencia y menor acoplamiento.



```
public class CalculadoraInline {  
  
    public double calcularPrecioFinal(double precio) {  
        return precio + precio * 0.16;  
        // Se eliminó el método innecesario  
    }  
  
}
```

Ejemplo: extraer método

```
public class ReporteVentas {  
    public void generarReporte() {  
        System.out.println("=== Reporte de Ventas ===");  
  
        // Cálculo del total de ventas  
        double totalVentas = 0;  
        double[] ventas = {100, 200, 150, 300};  
  
        for (double venta : ventas) {  
            totalVentas += venta;  
        }  
  
        System.out.println("Total de ventas: $" + totalVentas);  
  
        // Determinar si hay bonificación  
        if (totalVentas > 500) {  
            System.out.println("¡Felicidades! Se otorga una bonificación.");  
        }  
    }  
}
```



- ▶ generarReporte() ahora es más claro y solo organiza el flujo de ejecución.
- ▶ calcularTotalVentas() y verificarBonificacion() pueden ser usados en otros métodos.
- ▶ si queremos cambiar la lógica de ventas o bonificaciones, lo hacemos en su método específico.

```
public class ReporteVentas {  
    private void verificarBonificacion(double totalVentas) {  
        if (totalVentas > 500) {  
            System.out.println("¡Felicidades!  
            Se otorga una bonificación.");  
        }  
    }  
  
    private double calcularTotalVentas(double[] ventas) {  
        double total = 0;  
        for (double venta : ventas) {  
            total += venta;  
        }  
        return total;  
    }  
  
    public void generarReporte() {  
        System.out.println("=== Reporte de Ventas ===");  
  
        double[] ventas = {100, 200, 150, 300};  
        double totalVentas = calcularTotalVentas(ventas);  
  
        System.out.println("Total de ventas: $" + totalVentas);  
        verificarBonificacion(totalVentas);  
    }  
}
```

Ejemplo: extraer método en ECLIPSE

Reorganizamos un poco el código para poder usar la opción Extract Method de Eclipse:

```
package refactorizacion;

public class ReporteVentas {

    public void generarReporte() {
        System.out.println("=== Reporte de Ventas ===");

        // Cálculo del total de ventas

         double[] ventas = {100, 200, 150, 300};
        double totalVentas = 0;
        for (double venta : ventas) {
            totalVentas += venta;
        }

        System.out.println("Total de ventas: $" + totalVentas);

        // Determinar si hay bonificación
        if (totalVentas > 500) {
            System.out.println("¡Felicidades! Se otorga una bonificación.");
        }
    }
}
```

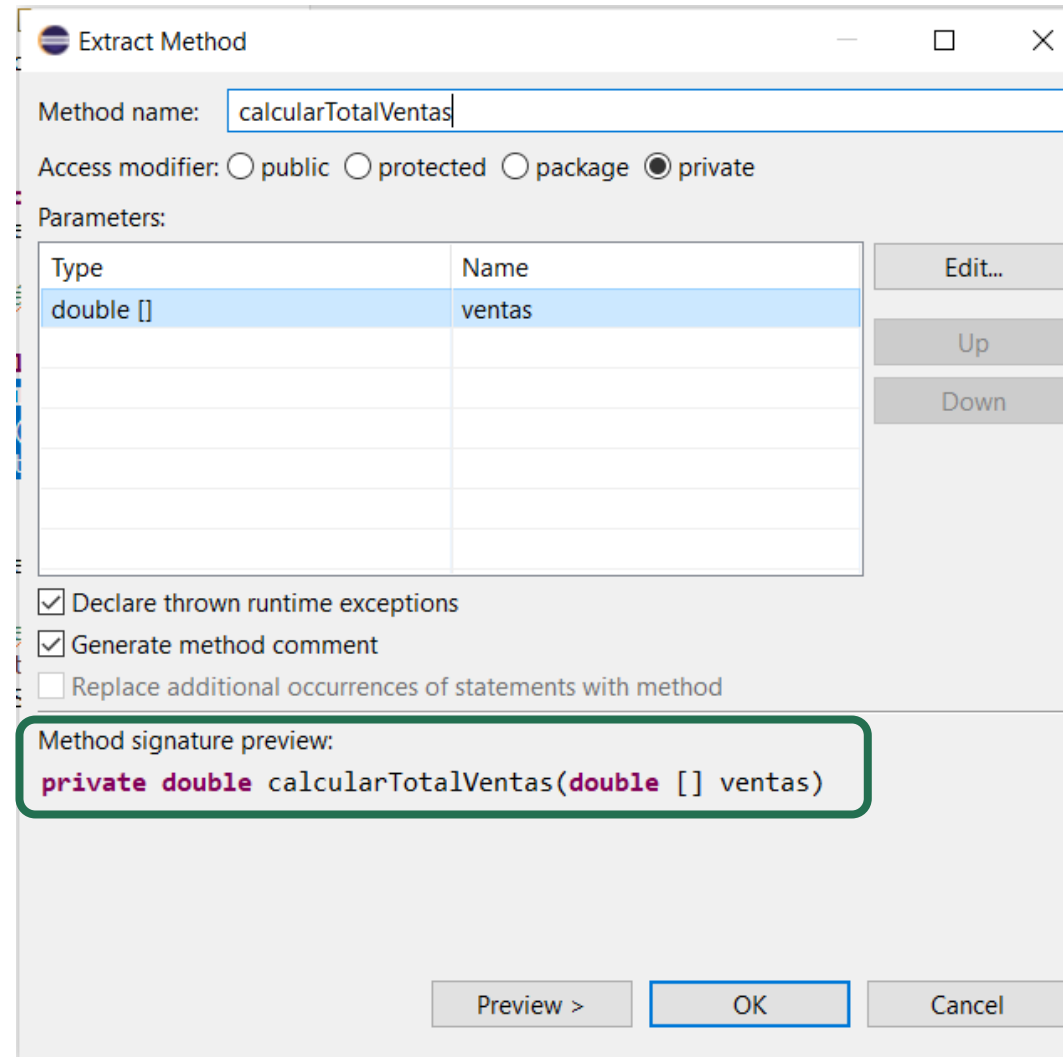
Ejemplo: extraer método en ECLIPSE

```
1 package refactorizacion;
2
3 public class ReporteVentas {
4
5     public void generarReporte() {
6         System.out.println("=== Reporte de Ventas ===");
7
8         // Cálculo del total de
9
10        double[] ventas = {100,
11        double totalVentas = 0;
12        for (double venta : ventas)
13            totalVentas += venta;
14    }
15
16    System.out.println("Total de Ventas: " + totalVentas);
17
18    // Determinar si hay bonificación
19    if (totalVentas > 500) {
20        System.out.println("Bonificación.");
21    }
22 }
23
24 }
```

The screenshot shows the Eclipse IDE with a Java file named 'ReporteVentas.java'. The code defines a class 'ReporteVentas' with a method 'generarReporte()' and a variable 'totalVentas'. A context menu is open over the code, showing various actions. The 'Refactor' option is selected, and a sub-menu is displayed with 'Extract Method...' highlighted.

Action	Shortcut
Undo Typing	Ctrl+Z
Revert File	
Save	Ctrl+S
Open Declaration	F3
Open Type Hierarchy	F4
Open Call Hierarchy	Ctrl+Alt+H
Show in Breadcrumb	Alt+Shift+B
Quick Outline	Ctrl+O
Quick Type Hierarchy	Ctrl+T
Open With	>
Show In	Alt+Shift+W >
Cut	Ctrl+X
Copy	Ctrl+C
Copy Qualified Name	
Paste	Ctrl+V
Raw Paste	
Quick Fix	Ctrl+1
Source	Alt+Shift+S >
Refactor	Alt+Shift+T >
Surround With	Alt+Shift+Z >
Local History	>
Move...	Alt+Shift+V
Change Method Signature...	Alt+Shift+C
Extract Method...	Alt+Shift+M

Ejemplo: extraer método en ECLIPSE



Extract Method

Method name:

Access modifier: ☐ public ☐ protected ☐ package ☒ private

Parameters:

Type	Name
double []	ventas

☒ Declare thrown runtime exceptions
☒ Generate method comment
☐ Replace additional occurrences of statements with method

Method signature preview:
private double calcularTotalVentas(**double []** ventas)

Ejemplo: extraer método en ECLIPSE

Preview:

The screenshot shows the Eclipse IDE interface during a refactoring operation. The 'Extract Method' dialog is open, showing the changes to be performed. The 'Original Source' and 'Refactored Source' panels are visible, showing the transformation of the code.

Changes to be performed

- ☒ ReporteVentas.java - Tema4/src/refactorizacion
 - ☒ Organize Imports
 - ☒ ReporteVentas
 - ☒ generarReporte()
 - ☒ Substitute statements with call to calcularTotalVentas
 - ☒ Create new method 'calcularTotalVentas' from selected statements

Original Source

```
1 public class ReporteVentas {
2
3     public void generarReporte() {
4         System.out.println("=== Reporte de Ventas ===");
5
6         // Cálculo del total de ventas
7
8         double[] ventas = {100, 200, 150, 300};
9         double totalVentas = 0;
10        for (double venta : ventas) {
11            totalVentas += venta;
12        }
13
14        System.out.println("Total de ventas: $" + totalVentas);
15
16        // Determinar si hay bonificación
17        if (totalVentas > 500) {
18            System.out.println("¡Felicidades! Se otorga un descuento");
19        }
20    }
21 }
22
23
24
25
```

Refactored Source

```
11 double totalVentas = calcularTotalVentas(ventas);
12
13 System.out.println("Total de ventas: $" + totalVentas);
14
15 // Determinar si hay bonificación
16 if (totalVentas > 500) {
17     System.out.println("¡Felicidades! Se otorga un descuento");
18 }
19
20 /**
21  * @param ventas
22  * @return
23  */
24 private double calcularTotalVentas(double[] ventas) {
25     double totalVentas = 0;
26     for (double venta : ventas) {
27         totalVentas += venta;
28     }
29     return totalVentas;
30 }
31
32 }
```

Ejemplo: extraer método en ECLIPSE

Resultado:

```
Empleado.java  *ReporteVentas.java x
1 package refactorizacion;
2
3 public class ReporteVentas {
4
5     public void generarReporte() {
6         System.out.println("=== Reporte de Ventas ===");
7
8         // Cálculo del total de ventas
9
10        double[] ventas = {100, 200, 150, 300};
11        double totalVentas = calcularTotalVentas(ventas);
12
13        System.out.println("Total de ventas: $" + totalVentas);
14
15        // Determinar si hay bonificación
16        if (totalVentas > 500) {
17            System.out.println("¡Felicidades! Se otorga una bonificación.");
18        }
19    }
20
21    /**
22     * @param ventas
23     * @return
24     */
25    private double calcularTotalVentas(double[] ventas) {
26        double totalVentas = 0;
27        for (double venta : ventas) {
28            totalVentas += venta;
29        }
30        return totalVentas;
31    }
32 }
```